

Chaos Engineering y la Pila Híbrida: Resiliencia Extrema en el Compilador

Estantería: DevSecOps e Infraestructura

Subtema: Chaos Engineering

mercedev.es — 2026-06-15 | Epic 9

Chaos Engineering y la Pila Híbrida: Resiliencia Extrema en el Compilador

En el ecosistema **mercedev.es**, el uso de la Inteligencia Artificial (IA) generativa es intensivo pero estrictamente regulado. Los agentes de IA (el "Lóbulo Frontal" o `merci-brain`) inyectan respuestas dinámicas y generativas en tiempo de compilación para que el sitio público final conserve latencia 0ms (Total Blocking Time 0ms) y cero dependencias de bases de datos.

Sin embargo, ¿qué sucede cuando la IA falla?

El Desafío: El Motor de IA como Punto Único de Fallo (SPOF)

La evolución de la arquitectura introdujo un dilema crítico: Si el servidor local de IA (Ollama) colapsa, o si la API remota de Google (Gemini) cambia sus políticas, el compilador del sitio (`merci-publish`) podría quedarse bloqueado esperando respuesta, estrellando todo el pipeline de Despliegue Continuo (CI/CD).

Era inaceptable tener un sistema de despliegue frágil. Se necesitaba certificar empíricamente la robustez.

La Maniobra: Pila Híbrida, Circuit Breaker y Chaos Monkey

Se diseñó una estrategia de resiliencia dividida en tres frentes tácticos:

1. El Patrón "Pila Híbrida" (Hybrid Stack)

Los agentes como `merci-brain.py` y `merci-blogger.py` se refactorizaron para no depender de un único proveedor. - **Intento Primario:** Se prioriza llamar al motor local y gratuito (Ollama: `qwen2.5-coder`). Si funciona, el coste es cero y la privacidad es total. - **Red de Seguridad (Fallback):** Si el agente detecta que Ollama no responde en un *timeout* estricto de 10 segundos, asume la caída y redirige inmediatamente el tráfico de red hacia el IDE Antigravity / Gemini Proxy (la nube) usando `litellm`.

2. El Patrón "Circuit Breaker"

La red de seguridad en la nube (Gemini) resolvió las caídas del servidor local, pero las APIs gratuitas imponen límites severos (*Rate Limiting*). Si el sistema intenta compilar 100 artículos y Gemini bloquea tras 8 peticiones con un error `HTTP 429 - Too Many Requests`, el pipeline volvería a romperse.

Para evitarlo, se implementó el **Circuit Breaker** (Cortacircuitos). Cuando `merci-brain.py` detecta un fallo de conexión persistente (tanto de Ollama como de Gemini), el sistema "corta" la electricidad: suspende las peticiones HTTP y aplica una contingencia silenciosa. Automáticamente inyecta una cadena de escape (`[Fallback]`) en el resto de artículos. El resultado final: la web se compila hasta el final con *Exit Code 0* y cero interrupciones de servicio.

3. El Juez: Chaos Engineering

Para certificar que estas defensas de papel funcionaban en el mundo real, recurrimos a **Chaos Engineering**. El script `merci-chaos.py` (*Chaos Monkey*) inyectó un sabotaje de red masivo (Táctica B): modificó dinámicamente las variables de entorno de Ollama para apuntar al puerto 9999 (un puerto muerto).

Aprendizaje y Deuda: El Caos como Auditor

La simulación de Caos reveló dos hitos clave:

1. **La revelación de la vulnerabilidad invisible:** En el primer ataque del Chaos Monkey, el *Fallback* saltó para salvar el sistema conectando con la nube. Sin embargo, falló miserablemente porque Google había obsoleto el modelo `gemini-1.5-flash` de su API v1beta y nosotros no lo sabíamos. Gracias al *Chaos Engineering*, este fallo silencioso fue expuesto en el entorno de desarrollo y lo parcheamos inmediatamente actualizando el ecosistema a `gemini-2.5-flash`.
2. **Triunfo total de Resiliencia:** Tras el parcheo, repetimos el ataque. El sistema interceptó la caída del motor local, conectó con Gemini, sufrió un estrangulamiento de red por el *Rate Limit* de Google, activó el *Circuit Breaker* bloqueando las peticiones, inyectó las medidas de contingencia en 84 artículos, y finalizó con un éxito del 100% (código 0). Todo el repositorio fue auto-curado de vuelta a su estado normal.

El objetivo de SRE no es que el sistema nunca falle; es que cuando todos sus componentes están ardiendo simultáneamente, el resultado para el usuario siga siendo inquebrantable.

Merci Explica: *En SRE, el Chaos Engineering no se trata de romper cosas a lo loco, sino de orquestar un "incendio controlado" para estudiar cómo actúan los aspersores. Descubrir que tu red de seguridad estaba podrida por un cambio de API antes de que llegue el fallo real en producción, es exactamente el momento en el que el caos demuestra su inmenso valor.*